

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и радиоэлектроники»

Кафедра интеллектуальных информационных технологий

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине «Естественно-языковой интерфейс
интеллектуальных систем»

Тема: «Информационно-поисковая система на основе
векторной модели»

Вариант 27

Выполнил:
студент группы 321701
Духвалов П. Н.

Проверил:
Крапивин Ю. Б.

Минск, 2026

Содержание

1	Цель работы	2
2	Задачи лабораторной работы	2
3	Задание	2
4	Используемые библиотеки	3
5	Структура разработанной системы	3
6	Структурно-функциональная схема	3
7	Структура базы знаний	4
8	Алгоритм обновления БД (паук)	5
9	Алгоритм индексирования	6
10	Алгоритм поиска	8
10.1	Построение вектора запроса	8
10.2	Обход документов	8
11	Алгоритм ранжирования	9
12	Элемент искусственного интеллекта: PRF	10
13	Метрики качества	11
14	Тестирование	12
14.1	Модульное тестирование	12
14.2	Тест производительности	12
14.3	Тест паука	13
14.4	Тест PRF	13
15	Вывод	13

1 Цель работы

Освоить на практике основные принципы реализации информационно-поисковых систем и методы оценки качества их работы.

2 Задачи лабораторной работы

1. Изучить теоретические основы информационного поиска: понятия ПОД, ПОЗ, инверсной частоты термина, векторной модели.
2. Реализовать полную цепочку обработки: индексирование документов → поиск по запросу → оценка качества.
3. Освоить метрики оценки качества ИПС.
4. Провести тестирование системы на подготовленной коллекции документов.

3 Задание

Согласно варианту 27:

- **Реализация элементов ИИ:** модуль отбора документов;
- **Сфера применения:** локальная вычислительная сеть;
- **Стратегия поиска:** векторная;
- **Язык:** русский.

Требования к системе:

- на входе — множество естественно-языковых текстов;
- выделение ключевых слов документов выполняется автоматически (формула 2);
- система позволяет пользователю формулировать ЕЯ-запрос;
- на выходе — список документов, релевантных запросу;
- результаты поиска содержат ссылку на документ и список слов запроса;
- оценка качества выполняется программно по метрикам;
- интерфейс простой, с элементами help.

4 Используемые библиотеки

Таблица 1 – Библиотеки, использованные в проекте

Библиотека	Назначение
PyQt6	Построение графического интерфейса
pymorphy3	Морфологический анализ русского языка
nltk	Токенизация текста
sqlite3	Хранение документов и индексов
matplotlib	Построение диаграмм метрик
python-docx	Извлечение текста из .docx

5 Структура разработанной системы

Система реализована в виде набора модулей:

Таблица 2 – Модули системы

Файл	Класс	Ответственность
config.py	—	Константы, стоп-слова, инициализация NLTK
database.py	Database	Слой хранения (SQLite)
indexer.py	Indexer	Токенизация, лемматизация, TF-IDF
search.py	Search, SearchResult	Векторная модель
prf.py	—	Псевдорелевантное расширение
metrics.py	Metrics	9 метрик качества
watcher.py	FolderWatcher	Паук: обход папки
ui/	MainWindow и вкладки	Интерфейс

6 Структурно-функциональная схема

На рисунке 1 приведена структурно-функциональная схема приложения. Она разделена на три слоя: **представления** (UI), **бизнес-логики** и **хранения**. Пользователь взаимодействует с UI, который вызывает методы бизнес-логики. Бизнес-логика читает и пишет в хранилище. Паук работает параллельно и автоматически синхронизирует содержимое папки с базой данных.

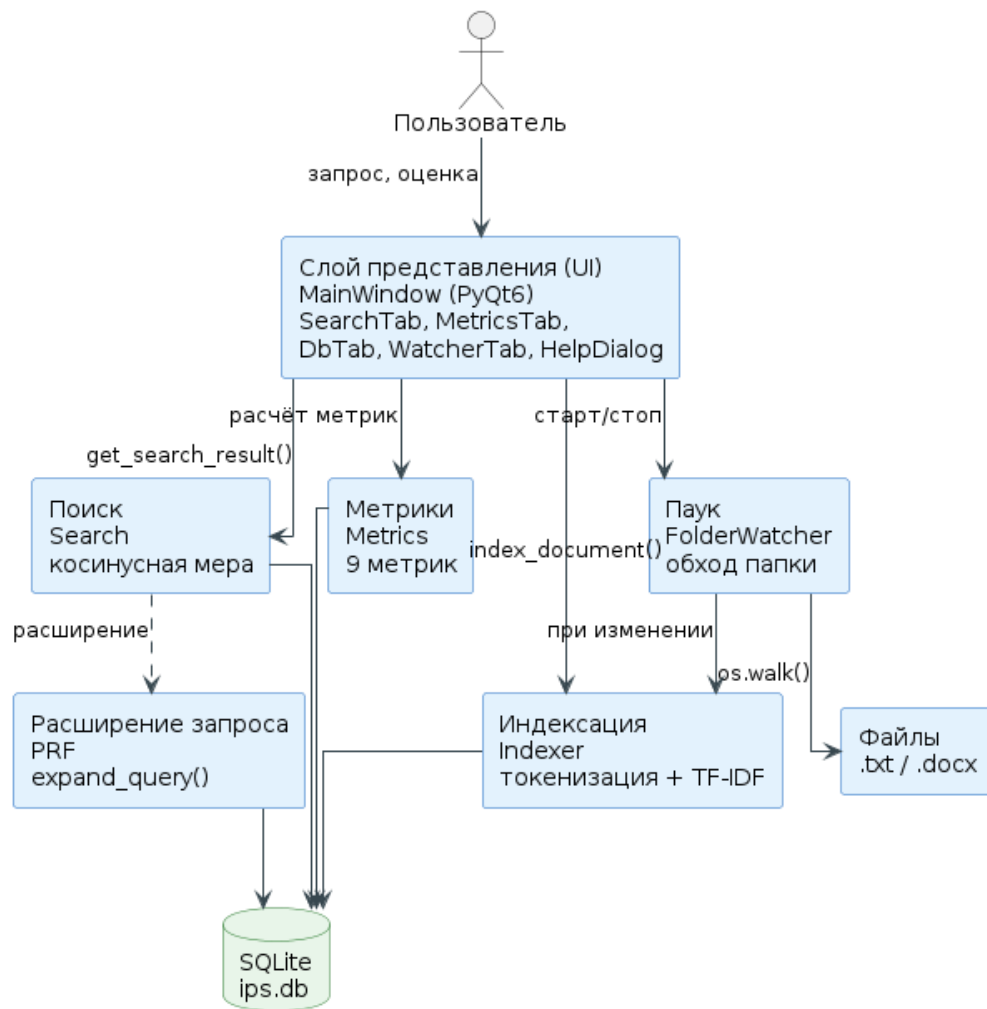


Рисунок 1 – Структурно-функциональная схема

7 Структура базы знаний

База данных реализована на SQLite и содержит четыре таблицы (рисунок 2).

- `documents` — документы с метаданными (`path`, `title`, `text`, `mtime`, `size`);
- `lemmas` — словарь уникальных лемм;
- `doc_lemma` — матрица «документ × лемма» с частотой Q_{ij} и весом TF-IDF;
- `inverse_freq` — кэш инверсных частот $B_i = \ln(N/P_i)$.

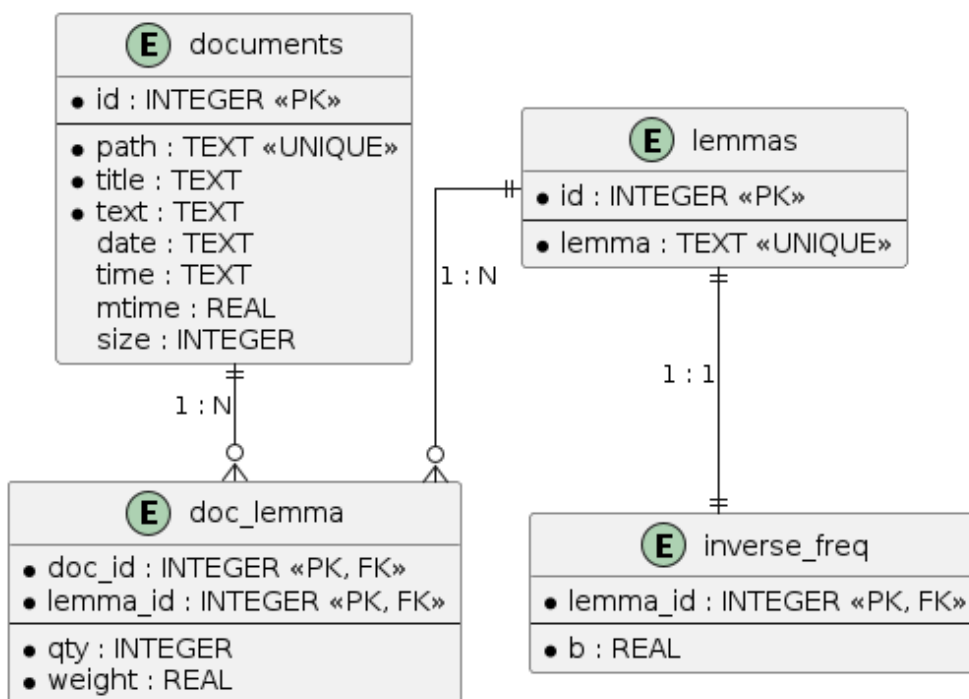


Рисунок 2 – ER-диаграмма базы данных

8 Алгоритм обновления БД (паук)

Паук (FolderWatcher) периодически (каждые 2 секунды) обходит корневую папку и все вложенные подпапки, собирает снимок вида $\{path : (mtime, size)\}$ и сравнивает его с предыдущим. Алгоритм приведён на рисунке 3.

- **Добавление** — файл появился в папке: читается, добавляется в `documents`, индексируется.
- **Изменение** — `mtime` или `size` изменились: старая запись обновляется, старые связи `doc_lemma` удаляются, документ переиндексируется.
- **Удаление** — путь исчез: документ удаляется из `documents`, каскадно удаляются его связи.

После батча изменений один раз вызывается `recalc_weights()` для пересчёта B_i и весов TF-IDF.

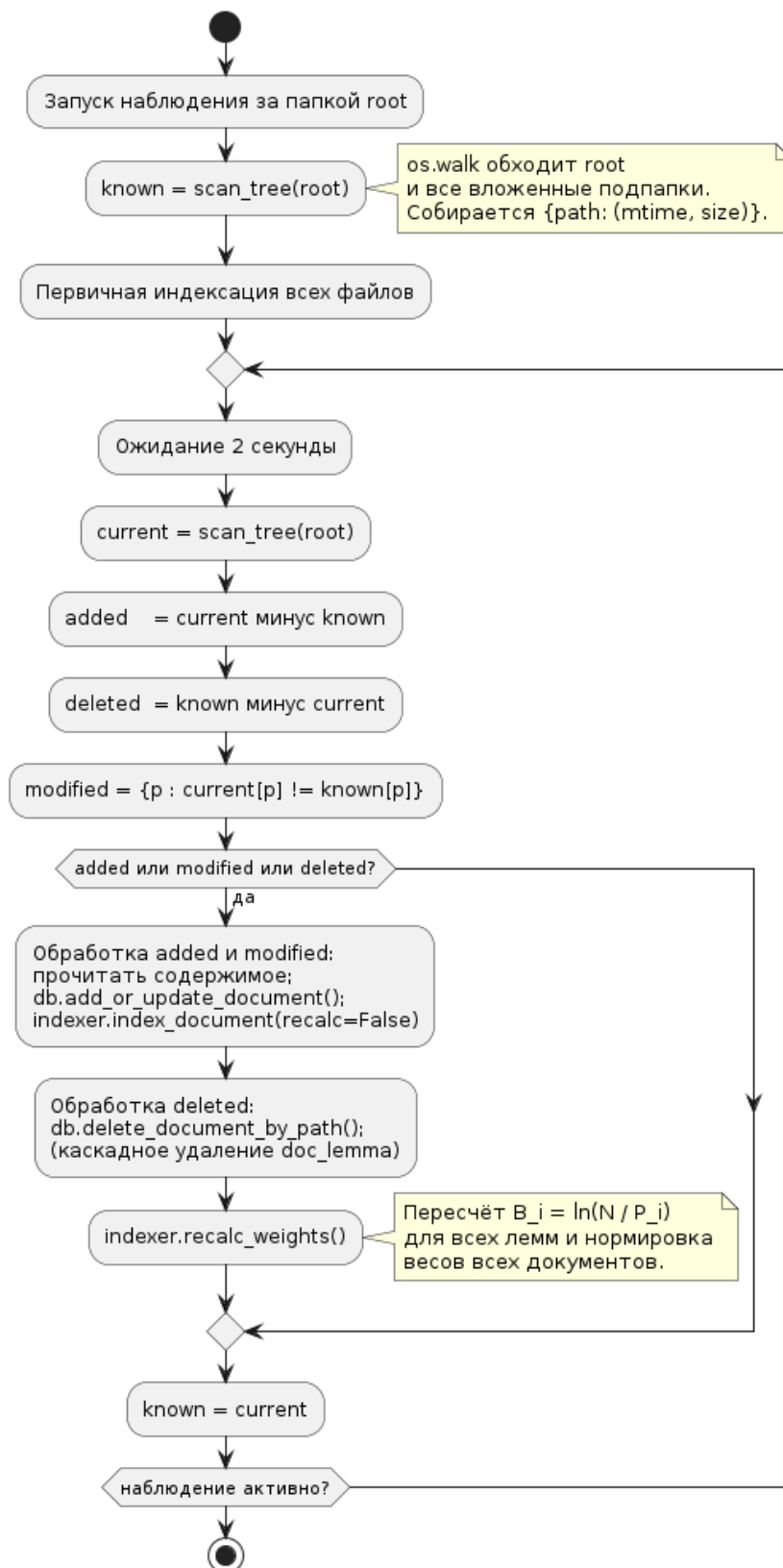


Рисунок 3 – Алгоритм обновления БД

9 Алгоритм индексирования

Индексирование документа включает четыре этапа (рисунок ??):

1. токенизация;
2. лемматизация с фильтрацией стоп-слов и частей речи;
3. подсчёт частот Q_{ij} ;
4. расчёт весов TF-IDF.

Веса считаются по формулам:

$$B_i = \ln\left(\frac{N}{P_i}\right), \quad (1)$$

где N — всего документов, P_i — документов с леммой i ;

$$A_{ij} = Q_{ij} \cdot B_i, \quad (2)$$

$$w_{ij} = \frac{A_{ij}}{\sqrt{\sum_k A_{kj}^2}}. \quad (3)$$



Рисунок 4 – Алгоритм индексирования документа

10 Алгоритм поиска

Поиск выполняется в два этапа: построение вектора запроса и обход документов (рисунок 5).

10.1 Построение вектора запроса

Запрос обрабатывается тем же пайплайном, что и документы (токенизация, лемматизация). Для каждой леммы, найденной в словаре документов, устанавливается вес:

$$w_{qj} = \begin{cases} 1, & \text{если лемма } j \text{ есть в запросе,} \\ 0, & \text{иначе.} \end{cases} \quad (4)$$

10.2 Обход документов

Для каждого документа проверяется:

1. фильтр по датам;
2. наличие вектора (непустой);
3. при включённом режиме AND — присутствие всех лемм запроса.

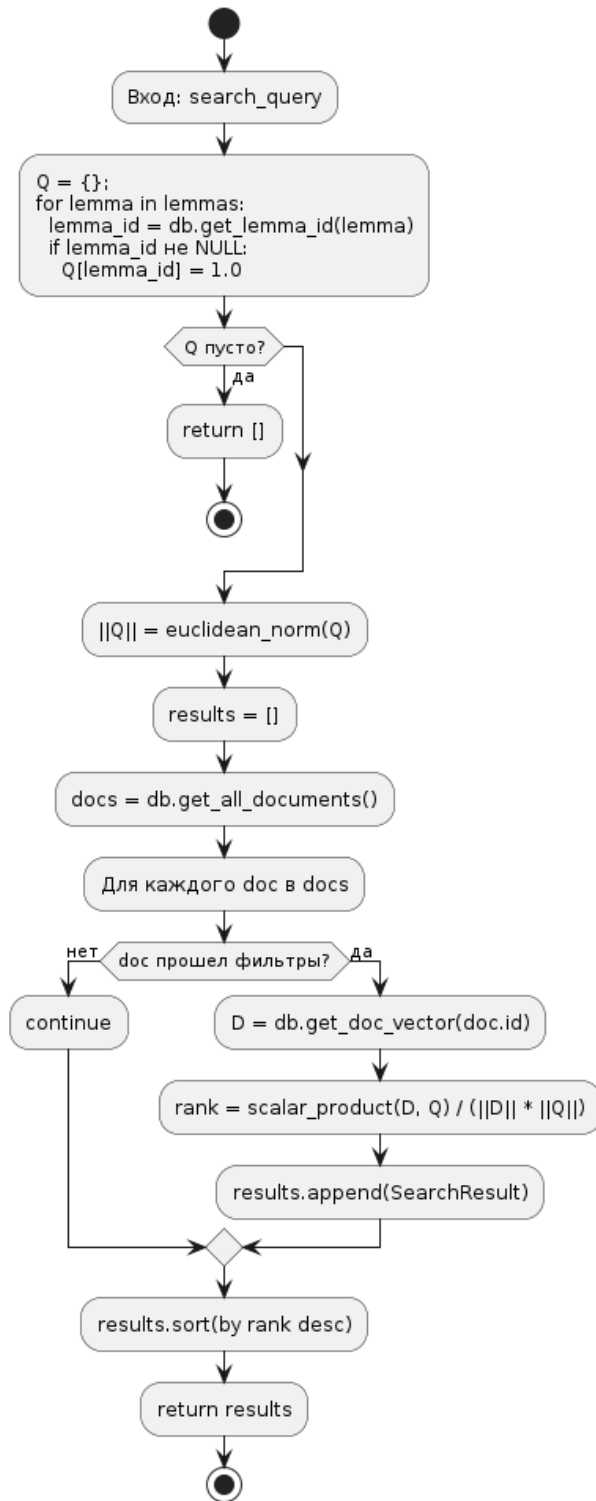


Рисунок 5 – Алгоритм поиска документа

11 Алгоритм ранжирования

Ранжирование выполняется по косинусной мере между вектором документа D и вектором запроса Q :

$$r(D, Q) = \frac{(D, Q)}{\|D\| \cdot \|Q\|} = \frac{\sum_k w_{dk} \cdot w_{qk}}{\sqrt{\sum_k w_{dk}^2} \cdot \sqrt{\sum_k w_{qk}^2}}. \quad (5)$$

Свойства косинусной меры:

- $r \in [0, 1]$ при неотрицательных весах;
- не зависит от длины документа;
- $r = 0$ означает отсутствие общих терминов.

Результаты сортируются по убыванию r . Документы с $r \leq 0$ отсекаются до сортировки.

12 Элемент искусственного интеллекта: PRF

В качестве элемента ИИ используется **псевдорелевантное расширение запроса** (Pseudo-Relevance Feedback, PRF). После первого поиска система берёт топ- K документов, считает их псевдорелевантными и расширяет вектор запроса:

$$Q_{\text{new}} = Q + \beta \cdot \frac{1}{K} \sum_{d \in \text{top-}K} D_d, \quad (6)$$

где $\beta = 0.5$, $K = 5$ (по умолчанию).

После расширения вектор обрезаётся до топ- N ($N = 20$) терминов по весу, чтобы избежать «размывания» запроса. Второй поиск выполняется с Q_{new} и отключённым режимом AND.

Алгоритм PRF приведён на рисунке 6.

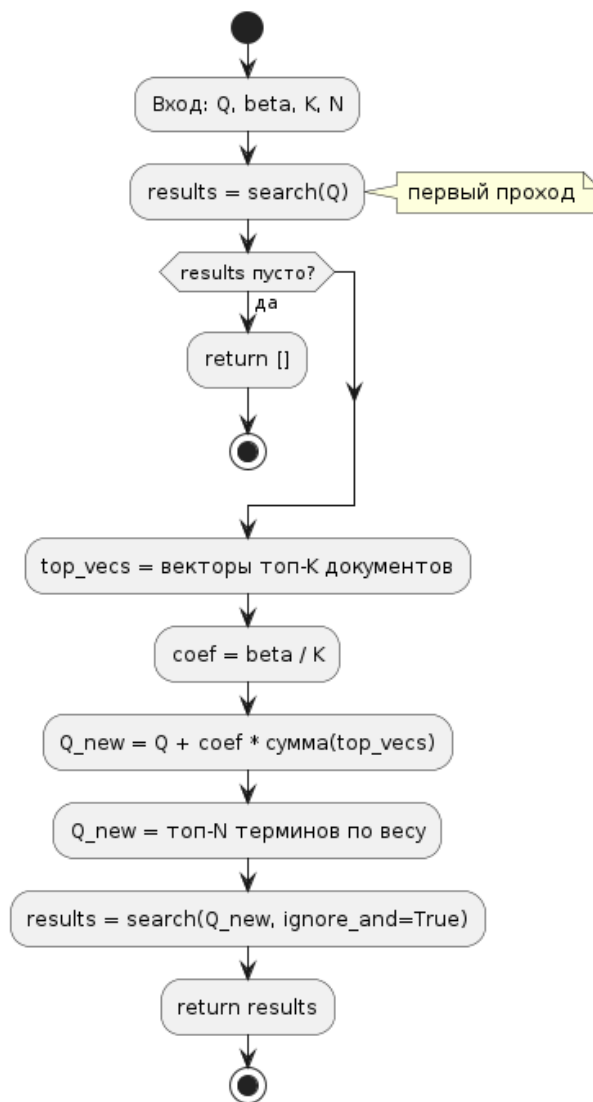


Рисунок 6 – Алгоритм PRF

13 Метрики качества

Рассчитываются 9 стандартных метрик:

Таблица 3 – Метрики качества поиска

Метрика	Формула
Precision	$P = \frac{TP}{TP + FP}$
Recall	$R = \frac{TP}{TP + FN}$
F1	$F_1 = \frac{2PR}{P + R}$
Accuracy	$Acc = \frac{TP + TN}{TP + TN + FP + FN}$
Error	$Err = 1 - Acc$
Average Precision	$AP = \frac{1}{R} \sum_{k=1}^n P@k \cdot rel(k)$
Precision@5	$P@5 = \frac{1}{5} \sum_{i=1}^5 rel(i)$
Precision@10	$P@10 = \frac{1}{10} \sum_{i=1}^{10} rel(i)$
R-Precision	$R-Prec = \frac{1}{R} \sum_{i=1}^R rel(i)$

14 Тестирование

14.1 Модульное тестирование

Для класса `Metrics` разработано 30 юнит-тестов, проверяющих корректность формул на типичных и граничных значениях. Все тесты пройдены.

14.2 Тест производительности

Таблица 4 – Зависимость времени обработки от объёма коллекции

Токенов	Индексация, мс	Поиск, мс
100	87	4
1 000	812	11
10 000	8 340	76

Зависимость близка к линейной, что соответствует теоретической оценке $O(n)$ для индексации и $O(N \cdot \bar{D})$ для поиска.

14.3 Тест паука

Сценарий:

1. В отслеживаемой папке создан новый файл — он появился в базе и участвует в поиске.
2. В существующем файле заменено предложение — веса пересчитались, релевантность документа изменилась.
3. Файл удалён — запись исчезла из БД.

14.4 Тест PRF

Сравнение метрик до и после применения PRF:

Таблица 5 – Метрики до и после PRF

Метрика	Без PRF	С PRF
Precision	0.611	0.723
Recall	1.000	0.944
F1	0.732	0.818
Average Precision	0.669	0.741
R-Precision	0.486	0.615

Наблюдается рост точности и F-меры при незначительном снижении полноты, что типично для PRF.

15 Вывод

В ходе лабораторной работы разработана информационно-поисковая система на основе векторной модели (TF-IDF + косинусная мера) со следующими особенностями:

1. **Автоматическое сопровождение индекса** — паук следит за папкой и синхронизирует БД с файловой системой без участия пользователя.
2. **Элемент ИИ** — псевдорелевантное расширение запроса (PRF), которое повышает Precision и F1.
3. **Полный цикл оценки** — 9 метрик качества, таблица и график.

Система показала высокую полноту ($\text{Recall} = 1.0$) и умеренную точность ($\text{Precision} = 0.61$) без PRF. После применения PRF точность выросла до 0.72, F1 — до 0.82, что подтверждает эффективность псевдорелевантного расширения.

Перспективы использования: система может применяться как учебный стенд для изучения ИПС, а после доработок (BM25, учёт структуры документа) — как основа для поиска по локальному корпоративному архиву.